

Kernelized sieve framework — formalization + Simulator B (δ -window multiplicity)

This document formalizes the **regularized filter-evolve transform** and gives a complete **reference implementation** of **Simulator B** using a **clustered-eigenvalue tolerance** (δ -window multiplicity) plus a **perturbation sweep** that yields a crisp **stability diagram**.

1) Core objects (typed)

Let $\mathcal{H}$ be a complex Hilbert space (numerically: $\mathbb{C}^N$).

1.1 Hamiltonian

A (bounded, finite-dimensional) Hermitian operator

$$\widehat{H} \in \mathbb{C}^{N \times N}, \quad \widehat{H} = \widehat{H}^\dagger.$$

1.2 Regularized kernelized evolution operator

Fix $\varepsilon > 0$. Let $K_\theta : [0, \infty) \rightarrow \mathcal{B}(\mathcal{H})$ be strongly measurable with

$$\int_0^\infty |K_\theta(\tau)| e^{-\varepsilon \tau} d\tau < \infty.$$

Define the bounded Bochner integral

$$\boxed{\alpha_\theta := \int_0^\infty K_\theta(\tau) e^{-(\varepsilon + \frac{i}{\hbar} \widehat{H})\tau} d\tau},$$

which satisfies

$$|\alpha_\theta| \leq \int_0^\infty |K_\theta(\tau)| e^{-\varepsilon \tau} d\tau.$$

1.3 Sieve dynamics (choose an interpretation)

(A) Post-selected / renormalized (projective) dynamics

$$\boxed{\psi_{t+1} = \frac{\mathcal{F}_\theta(t, U, \psi_t)}{\|\mathcal{F}_\theta(t, U, \psi_t)\|}}$$

with a unitary step $U = e^{-\frac{i}{\hbar} \widehat{H} \Delta t}$ or any other chosen unitary.

(B) CPTP open-system (physically honest filtering) Choose Kraus operators $\{M_j(\theta, t)\}$ such that $\sum_j M_j^\dagger M_j = I$, and set

$$\boxed{\rho_{t+1} = \sum_j M_j \rho_t U_j^\dagger, M_j^\dagger}$$

Simulator B below focuses on defining $\mathcal{F}_\theta$ (or Kraus families) from **δ -window multiplicity classes**.

2) Simulator B — multiplicity sieve with δ -window clustering

2.1 Motivation

Exact eigenvalue multiplicities are unstable: generic perturbations split degeneracies. So we replace “multiplicity” by **cluster size under a tolerance** $\delta > 0$.

2.2 Spectral decomposition

Let

$$\widehat{H} = V \mathrm{diag}(\lambda_1, \dots, \lambda_N) V^\dagger,$$

where V is unitary and $\lambda_1 \leq \dots \leq \lambda_N$ are real.

2.3 δ -clusters and δ -window multiplicity

Define clusters by grouping consecutive eigenvalues whenever adjacent gaps are small.

δ -cluster rule (gap-based): Let $\mathcal{C}_1, \dots, \mathcal{C}_m$ be a partition of $\{1, \dots, N\}$ into contiguous index sets such that:

- if $i, i+1 \in \mathcal{C}_k$ then $|\lambda_{i+1} - \lambda_i| \leq \delta$,
- if $i \in \mathcal{C}_k, i+1 \in \mathcal{C}_{k+1}$ then $|\lambda_{i+1} - \lambda_i| > \delta$.

The **δ -window multiplicity** of a cluster $\mathcal{C}$ is $|\mathcal{C}|$.

2.4 Cluster projectors and multiplicity gates

For each cluster $\mathcal{C} \subset \{1, \dots, N\}$, define the orthogonal projector onto its eigenspace span:

$$P_{\mathcal{C}} := \sum_{i \in \mathcal{C}} v_i v_i^\dagger \quad \text{(where } v_i \text{ are columns of } V \text{)}$$

For a target multiplicity $\kappa \in \{1, \dots, N\}$, define the **δ -window multiplicity projector**

$$\boxed{\Pi_{\{\kappa, \delta\}}(\widehat{H}) := \sum_{\mathcal{C}: |\mathcal{C}| = \kappa} P_{\mathcal{C}}}$$

This is an orthogonal projector (sum of orthogonal projectors onto disjoint eigenspaces).

2.5 The multiplicity sieve filter

A simple filter is

$$\mathcal{F}_{\kappa,\delta,\eta}(\widehat{H}) := (1-\eta)I + \eta \Pi_{\kappa,\delta}(\widehat{H}), \quad \eta \in [0,1].$$

- $\eta = 1$ is “hard” projection.
- $\eta < 1$ is a **nonexpansive soft filter** (useful if you want contraction-style stability claims).

3) Perturbation model + stability diagram

3.1 Designed baseline Hamiltonian with exact degeneracies

Construct $\widehat{H}_0$ with block-degeneracies (exact repeated eigenvalues). Example: diagonal with repeated values:

- one block of size κ at energy E_κ ,
- other blocks with different multiplicities and energies separated by large gaps.

3.2 Perturbations

Use a Hermitian random perturbation Δ with $\|\Delta\| \approx 1$ and sweep the perturbation strength $\sigma \geq 0$:

$$\widehat{H}(\sigma) = \widehat{H}_0 + \sigma \Delta.$$

3.3 Two stability metrics (use both)

Let $P_0 := \Pi_{\kappa,\delta}(\widehat{H}_0)$, $P(\sigma) := \Pi_{\kappa,\delta}(\widehat{H}(\sigma))$.

1. Subspace overlap (rank-normalized):

$$S_{\text{overlap}}(\sigma, \delta) := \frac{\text{tr}(P(\sigma)P_0)}{\max(1, \text{rank}(P_0))}.$$

Interpretable as “fraction of the original target subspace still captured.”

1. Random-state capture (averaged): draw $\psi \sim \text{Haar-ish}$ by normalizing a complex Gaussian vector; compute

$$S_{\text{mass}}(\sigma, \delta) := \mathbb{E}_{\psi} |P(\sigma)\psi|^2.$$

This measures “how much of a typical state lands in the current κ -class subspace.”

3.4 Stability diagram

For grids $\sigma \in [0, \sigma_{\max}]$ and $\delta \in [\delta_{\min}, \delta_{\max}]$, plot a heatmap of either metric. Expect a boundary curve:

- small σ and/or large $\delta \rightarrow \kappa$ -cluster survives $\rightarrow$ high stability,
- large σ and small $\delta \rightarrow \kappa$ -cluster splits $\rightarrow$ stability collapses.

4) Reference implementation (single Python script)

Copy/paste into a file, e.g. `simulator_B_stability.py`.

```
import numpy as np

# -----
# Utilities
# -----

def complex_gaussian(N: int, rng: np.random.Generator) -> np.ndarray:
    x = rng.normal(size=N) + 1j * rng.normal(size=N)
    x /= np.linalg.norm(x)
    return x

def random_hermitian(N: int, rng: np.random.Generator) -> np.ndarray:
    """Random Hermitian matrix with O(1) operator norm scale."""
    A = rng.normal(size=(N, N)) + 1j * rng.normal(size=(N, N))
    H = (A + A.conj().T) / 2.0
    # Normalize to have Frobenius norm ~ 1 (keeps sigma interpretation stable)
    H /= np.linalg.norm(H, ord='fro')
    return H

def make_H0_from_blocks(block_sizes, block_energies) -> np.ndarray:
    """Diagonal H0 with exact degeneracies defined by blocks."""
    assert len(block_sizes) == len(block_energies)
    diag = []
    for m, E in zip(block_sizes, block_energies):
        diag.extend([E] * m)
    diag = np.array(diag, dtype=float)
    return np.diag(diag)

# -----
#  $\delta$ -window clustering
# -----
```

```

def cluster_indices_by_gap(evals_sorted: np.ndarray, delta: float):
    """Return list of clusters (each a list of indices) based on adjacent gaps
    <= delta."""
    N = evals_sorted.size
    clusters = []
    start = 0
    for i in range(N - 1):
        if abs(evals_sorted[i + 1] - evals_sorted[i]) > delta:
            clusters.append(list(range(start, i + 1)))
            start = i + 1
    clusters.append(list(range(start, N)))
    return clusters

def multiplicity_projector(H: np.ndarray, kappa: int, delta: float):
    """
    Build  $\Pi_{\{\kappa, \delta\}}(H)$  as sum of projectors onto eigenspace clusters
    whose  $\delta$ -window multiplicity (cluster size) equals kappa.

    Returns:
        P (NxN projector), rank(P), evals_sorted
    """
    evals, evecs = np.linalg.eigh(H) # evals sorted ascending; evecs columns
    clusters = cluster_indices_by_gap(evals, delta)

    N = H.shape[0]
    P = np.zeros((N, N), dtype=complex)
    rank = 0

    for C in clusters:
        if len(C) == kappa:
            Vc = evecs[:, C] # N x kappa
            P += Vc @ Vc.conj().T
            rank += len(C)

    # Numerical symmetrization
    P = (P + P.conj().T) / 2.0
    return P, rank, evals

def overlap_metric(P: np.ndarray, P0: np.ndarray, rank0: int) -> float:
    # tr(P P0) is real for projectors; take real part defensively
    return float(np.real(np.trace(P @ P0)) / max(1, rank0))

def random_mass_metric(P: np.ndarray, N: int, rng: np.random.Generator, trials:
int = 20) -> float:

```

```

acc = 0.0
for _ in range(trials):
    psi = complex_gaussian(N, rng)
    acc += float(np.vdot(P @ psi, P @ psi).real) # ||P psi||^2
return acc / trials

# -----
# Perturbation sweep + stability diagram
# -----

def run_stability_sweep(
    N: int = 30,
    kappa: int = 3,
    block_sizes=(3, 2, 4, 5, 16),
    block_energies=(0.0, 20.0, 50.0, 90.0, 150.0),
    sigmas=np.linspace(0.0, 2.0, 41),
    deltas=np.linspace(1e-3, 0.5, 60),
    noise_trials: int = 20,
    mass_trials: int = 20,
    seed: int = 0,
):
    """
    Produces two heatmaps:
        S_overlap[delta_index, sigma_index]
        S_mass[delta_index, sigma_index]

    Interpretation:
        - S_overlap ~ 1: original  $\kappa$ -subspace is still captured under perturbation
        - S_overlap ~ 0:  $\kappa$ -class has drifted/split beyond recognition (w.r.t.  $\delta$ )

        - S_mass measures typical capture into current  $\kappa$ -class (can rise if  $\kappa$ -
        class appears elsewhere)
    """
    rng = np.random.default_rng(seed)

    # Baseline exact-degeneracy Hamiltonian
    H0 = make_H0_from_blocks(block_sizes, block_energies)
    assert H0.shape == (N, N)

    # Precompute baseline projector for each delta (useful because P0 depends on
    delta)
    P0_by_delta = []
    rank0_by_delta = []
    for delta in deltas:
        P0, r0, _ = multiplicity_projector(H0, kappa, float(delta))
        P0_by_delta.append(P0)

```

```

        rank0_by_delta.append(r0)

S_overlap = np.zeros((len(deltas), len(sigmas)), dtype=float)
S_mass = np.zeros((len(deltas), len(sigmas)), dtype=float)

for j_sigma, sigma in enumerate(sigmas):
    # Average over independent perturbations at this sigma
    overlap_acc = np.zeros(len(deltas), dtype=float)
    mass_acc = np.zeros(len(deltas), dtype=float)

    for _ in range(noise_trials):
        Delta = random_hermitian(N, rng)
        H = H0 + float(sigma) * Delta

        for i_delta, delta in enumerate(deltas):
            P, _, _ = multiplicity_projector(H, kappa, float(delta))
            P0 = P0_by_delta[i_delta]
            r0 = rank0_by_delta[i_delta]

            overlap_acc[i_delta] += overlap_metric(P, P0, r0)
            mass_acc[i_delta] += random_mass_metric(P, N, rng,
            trials=mass_trials)

    S_overlap[:, j_sigma] = overlap_acc / noise_trials
    S_mass[:, j_sigma] = mass_acc / noise_trials

return {
    "H0": H0,
    "sigmas": np.array(sigmas, dtype=float),
    "deltas": np.array(deltas, dtype=float),
    "S_overlap": S_overlap,
    "S_mass": S_mass,
    "kappa": kappa,
    "block_sizes": block_sizes,
    "block_energies": block_energies,
    "noise_trials": noise_trials,
    "mass_trials": mass_trials,
}

def plot_heatmap(X, y_vals, x_vals, title, xlabel="sigma", ylabel="delta"):
    import matplotlib.pyplot as plt

    plt.figure()
    plt.imshow(
        X,
        aspect='auto',
        origin='lower',

```

```

        extent=[x_vals[0], x_vals[-1], y_vals[0], y_vals[-1]],
    )
    plt.colorbar()
    plt.title(title)
    plt.xlabel(xlabel)
    plt.ylabel(ylabel)
    plt.show()

if __name__ == "__main__":
    out = run_stability_sweep(
        N=30,
        kappa=3,
        block_sizes=(3, 2, 4, 5, 16),
        block_energies=(0.0, 20.0, 50.0, 90.0, 150.0),
        sigmas=np.linspace(0.0, 2.0, 41),
        deltas=np.linspace(1e-3, 0.5, 60),
        noise_trials=15,
        mass_trials=15,
        seed=0,
    )

    plot_heatmap(
        out["S_overlap"],
        out["deltas"],
        out["sigmas"],
        title=f"Simulator B stability (overlap)  κ={out['kappa']}"
    )

    plot_heatmap(
        out["S_mass"],
        out["deltas"],
        out["sigmas"],
        title=f"Simulator B stability (random-state mass)  κ={out['kappa']}"
    )

```

5) What to expect (qualitative)

- For very small σ , the κ -degenerate block in H_0 remains nearly κ -clustered, so for any δ larger than the induced splitting you should see **high overlap**.
- As σ increases, the κ -block splits; for **fixed δ** , overlap drops sharply once typical adjacent gaps exceed δ .
- Increasing δ “forgives” splitting, shifting the stability boundary upward (stable for larger σ).

This produces the crisp **phase-boundary** you want: a curve in (σ, δ) separating “ κ -multiplicity recognized” from “ κ -multiplicity lost.”

6) Optional: connect to filter-evolve dynamics

Once you have $\Pi_{\kappa,\delta}(H(\sigma))$, you can embed it into the sieve iteration:

$$\psi_{t+1} = \frac{\big((1-\eta)I + \eta \Pi_{\kappa,\delta}(H(\sigma))\big) U \psi_t}{\|\cdot\|}.$$

A practical choice to emphasize noncommutation is to define U in a basis different from H 's eigenbasis (e.g., Fourier or random unitary), so that swapping UF vs FU produces measurable order sensitivity.

7) Filtered dynamics + convergence overlays on the (σ, δ) phase map

This section implements the projective filtered dynamics

$$\psi_{t+1} = \text{Normalize}(F_{\kappa,\delta,\eta}(H(\sigma)) \cdot U \cdot \psi_t)$$

and produces a **convergence-speed field** over the same (σ, δ) grid, which can be **overlaid as contour curves** on top of the stability heatmap.

7.1 Experiment definitions

- Perturbed Hamiltonian: $H(\sigma) = H_0 + \sigma\Delta$ with Δ Hermitian.
- δ -window κ -projector: $\Pi_{\kappa,\delta}(H(\sigma))$ built by δ -gap clustering of eigenvalues (Section 2).
- Soft filter: $F_{\kappa,\delta,\eta}(H(\sigma)) = (1-\eta)I + \eta \Pi_{\kappa,\delta}(H(\sigma))$, with $\eta \in [0,1]$.
- Noncommuting unitary step U : use an FFT-based unitary (typically does not commute with $\Pi_{\kappa,\delta}(H(\sigma))$).

7.2 Convergence quantities to overlay

For each (σ, δ) , run $t = 0, 1, \dots, T$ steps and record capture into a **reference κ -subspace**.

- Reference projector (baseline): $P_0(\delta) := \Pi_{\kappa,\delta}(H_0)$.
- Time series: $M_t(\sigma, \delta) := \|\Pi_{\kappa,\delta}(H(\sigma)) \psi_t - P_0(\delta) \psi_t\|^2 \in [0,1]$.

Define a robust tail estimate of the asymptote:

$$M_\infty(\sigma, \delta) := \text{median}\{M_t : t \in \{T-w, \dots, T\}\}.$$

Define a “time-to-fraction” convergence time (first passage):

$$T_{\text{frac}}(\sigma, \delta) := \min\{t : M_t \geq \text{frac} \cdot M_\infty\} \text{ with } \text{frac} \in (0,1).$$

Overlay idea: plot a stability heatmap (e.g., S_{overlap}) and overlay **contour curves** of T_{frac} in the (σ, δ) plane (fast vs slow convergence regions).

Notes:

- If $M_{\infty} \approx 0$ (no meaningful capture), set $T_{\text{frac}} := T$ by convention.
- If you prefer success relative to the *current* κ -subspace, replace $P_0(\delta)$ with $P(\sigma, \delta) := \Pi_{\{\kappa, \delta\}}(H(\sigma))$.

7.3 Code: dynamics sweep + contour overlay

Append the following to the end of the script in Section 4 (or place it in a second file that imports the earlier functions).

```
import numpy as np

# -----
# Unitary step U (FFT-based, typically noncommuting with H-eigenspaces)
# -----

def ufft(x: np.ndarray) -> np.ndarray:
    N = x.size
    return np.fft.fft(x) / np.sqrt(N)

def uifft(x: np.ndarray) -> np.ndarray:
    N = x.size
    return np.fft.ifft(x) * np.sqrt(N)

def make_U_fourier_log(N: int, dt: float = 0.7):
    """U = F* diag(exp(-i log n dt)) F."""
    n = np.arange(1, N + 1)
    phases = np.exp(-1j * np.log(n) * dt)

    def U(psi: np.ndarray) -> np.ndarray:
        return uifft(phases * ufft(psi))

    return U

def normalize(psi: np.ndarray) -> np.ndarray:
    nrm = np.linalg.norm(psi)
    return psi if nrm == 0 else psi / nrm
```

```

# -----
# Fast application of  $\Pi_{\{\kappa,\delta\}}(H)$  without building a full projector
matrix
# -----

def multiplicity_cluster_bases(H: np.ndarray, kappa: int, delta: float):
    """
    Returns a list of basis blocks  $V_c$  ( $N \times \kappa$ ), one per  $\delta$ -cluster of size
    kappa.
    Then  $\Pi \psi = \sum_c V_c (V_c^* \psi)$ .
    """
    evals, evecs = np.linalg.eigh(H)
    clusters = cluster_indices_by_gap(evals, delta)

    bases = []
    for C in clusters:
        if len(C) == kappa:
            bases.append(evecs[:, C])
    return bases

def apply_projector_bases(bases, psi: np.ndarray) -> np.ndarray:
    out = np.zeros_like(psi)
    for Vc in bases:
        out += Vc @ (Vc.conj().T @ psi)
    return out

# -----
# Filtered dynamics for a single (sigma, delta)
# -----

def run_filtered_dynamics(
    H0: np.ndarray,
    H: np.ndarray,
    kappa: int,
    delta: float,
    U,
    eta: float = 1.0,
    steps: int = 80,
    tail_window: int = 10,
    seed: int = 0,
):
    rng = np.random.default_rng(seed)

    N = H.shape[0]
    psi = rng.normal(size=N) + 1j * rng.normal(size=N)
    psi = normalize(psi)

```

```

# Reference projector P0(delta) (baseline)
bases0 = multiplicity_cluster_bases(H0, kappa, delta)

# Current  $\kappa$ -projector bases for H(sigma)
bases = multiplicity_cluster_bases(H, kappa, delta)

def F_apply(x: np.ndarray) -> np.ndarray:
    #  $F = (1-\eta)I + \eta \Pi$ 
    if eta == 0.0:
        return x
    if len(bases) == 0:
        return (1.0 - eta) * x
    Px = apply_projector_bases(bases, x)
    return (1.0 - eta) * x + eta * Px

M = []
for _ in range(steps):
    psi = normalize(F_apply(U(psi)))
    if len(bases0) == 0:
        M.append(0.0)
    else:
        P0psi = apply_projector_bases(bases0, psi)
        M.append(float(np.vdot(P0psi, P0psi).real))

M = np.array(M, dtype=float)
w = min(tail_window, steps)
M_inf = float(np.median(M[-w:]))
return M, M_inf

# -----
# Dynamics sweep: build convergence-time field T_frac(sigma, delta)
# -----

def run_dynamics_sweep(
    H0: np.ndarray,
    kappa: int,
    sigmas: np.ndarray,
    deltas: np.ndarray,
    dt: float = 0.7,
    eta: float = 1.0,
    steps: int = 80,
    frac: float = 0.9,
    noise_trials: int = 10,
    seed: int = 0,
):
    rng = np.random.default_rng(seed)

```

```

N = H0.shape[0]
U = make_U_fourier_log(N, dt=dt)

T_field = np.full((len(deltas), len(sigmas)), float(steps), dtype=float)
M_inf_field = np.zeros((len(deltas), len(sigmas)), dtype=float)

for j, sigma in enumerate(sigmas):
    T_acc = np.zeros(len(deltas), dtype=float)
    M_acc = np.zeros(len(deltas), dtype=float)

    for trial in range(noise_trials):
        Delta = random_hermitian(N, rng)
        H = H0 + float(sigma) * Delta

        for i, delta in enumerate(deltas):
            M, M_inf = run_filtered_dynamics(
                H0=H0,
                H=H,
                kappa=kappa,
                delta=float(delta),
                U=U,
                eta=eta,
                steps=steps,
                tail_window=max(5, steps // 8),
                seed=seed + 1000 * trial + 17 * i + 31 * j,
            )

            if M_inf <= 1e-12:
                t_hit = steps
            else:
                thresh = frac * M_inf
                hits = np.nonzero(M >= thresh)[0]
                t_hit = int(hits[0]) if hits.size > 0 else steps

            T_acc[i] += t_hit
            M_acc[i] += M_inf

    T_field[:, j] = T_acc / noise_trials
    M_inf_field[:, j] = M_acc / noise_trials

return {"T_frac": T_field, "M_inf": M_inf_field, "dt": dt, "eta": eta,
        "steps": steps, "frac": frac}

# -----
# Plot: stability heatmap + convergence contours
# -----

```

```

def plot_stability_with_convergence(
    S_stability: np.ndarray,
    T_frac: np.ndarray,
    deltas: np.ndarray,
    sigmas: np.ndarray,
    title: str,
    contour_levels=(5, 10, 20, 30, 40, 60, 80),
):
    import matplotlib.pyplot as plt

    plt.figure()
    plt.imshow(
        S_stability,
        aspect='auto',
        origin='lower',
        extent=[sigmas[0], sigmas[-1], deltas[0], deltas[-1]],
    )
    plt.colorbar(label="stability")

    X, Y = np.meshgrid(sigmas, deltas)
    cs = plt.contour(X, Y, T_frac, levels=contour_levels)
    plt.clabel(cs, inline=True, fontsize=8, fmt="%d")

    plt.title(title)
    plt.xlabel("sigma")
    plt.ylabel("delta")
    plt.show()

# -----
# Example main: compute stability + dynamics and overlay
# -----

if __name__ == "__main__":
    out = run_stability_sweep(
        N=30,
        kappa=3,
        block_sizes=(3, 2, 4, 5, 16),
        block_energies=(0.0, 20.0, 50.0, 90.0, 150.0),
        sigmas=np.linspace(0.0, 2.0, 41),
        deltas=np.linspace(1e-3, 0.5, 60),
        noise_trials=10,
        mass_trials=10,
        seed=0,
    )

    dyn = run_dynamics_sweep(
        H0=out["H0"],

```

```

        kappa=out["kappa"],
        sigmas=out["sigmas"],
        deltas=out["deltas"],
        dt=0.7,
        eta=1.0,
        steps=80,
        frac=0.9,
        noise_trials=6,
        seed=1,
    )

    plot_stability_with_convergence(
        S_stability=out["S_overlap"],
        T_frac=dyn["T_frac"],
        deltas=out["deltas"],
        sigmas=out["sigmas"],
        title=f"Overlap stability with convergence contours
(kappa={out['kappa']}, frac={dyn['frac']})",
        contour_levels=(5, 10, 20, 30, 40, 60, 80),
    )

```

7.4 Optional: raw convergence curves at selected (σ , δ)

```

import matplotlib.pyplot as plt

def plot_convergence_examples(H0, kappa, points, dt=0.7, eta=1.0, steps=80,
seed=0):
    rng = np.random.default_rng(seed)
    N = H0.shape[0]
    U = make_U_fourier_log(N, dt=dt)

    plt.figure()
    for idx, (sigma, delta) in enumerate(points):
        Delta = random_hermitian(N, rng)
        H = H0 + float(sigma) * Delta
        M, M_inf = run_filtered_dynamics(
            H0=H0, H=H, kappa=kappa, delta=float(delta), U=U,
            eta=eta, steps=steps, tail_window=max(5, steps // 8), seed=seed +
123 * idx
        )
        plt.plot(M, label=f"sigma={sigma:.2f}, delta={delta:.3f}, M_inf={M_inf:.
2f}")

    plt.xlabel("step t")
    plt.ylabel("M_t = ||P0(delta) psi_t||^2")
    plt.title("Filtered-dynamics convergence examples")

```

```
plt.legend(fontsize=8)
plt.show()

# Example usage:
# plot_convergence_examples(out["H0"], out["kappa"], points=[(0.1, 0.02), (1.2,
0.02), (1.2, 0.25)])
```

8) References (for the article)

Spectral theory / functional calculus / semigroups

1. M. Reed and B. Simon, Methods of Modern Mathematical Physics, Vol. I: Functional Analysis.
2. M. Reed and B. Simon, Methods of Modern Mathematical Physics, Vol. IV: Analysis of Operators.
3. T. Kato, Perturbation Theory for Linear Operators.
4. E. B. Davies, Linear Operators and their Spectra.

Matrix analysis / perturbation bounds / subspace angles 5. R. Bhatia, Matrix Analysis. 6. G. W. Stewart and J.-G. Sun, Matrix Perturbation Theory. 7. C. Davis and W. M. Kahan, The rotation of eigenvectors by a perturbation (SIAM J. Numer. Anal., 1970).

Quantum operations / measurement / open systems 8. M. A. Nielsen and I. L. Chuang, Quantum Computation and Quantum Information. 9. H.-P. Breuer and F. Petruccione, The Theory of Open Quantum Systems. 10. J. Preskill, Lecture Notes on Quantum Computation.

Iterative methods / power-method intuition 11. Y. Saad, Iterative Methods for Sparse Linear Systems. 12. G. H. Golub and C. F. Van Loan, Matrix Computations.

Operator splitting / noncommutation diagnostics 13. H. F. Trotter, On the product of semi-groups of operators (Proc. AMS, 1959). 14. M. Suzuki, Higher-order decompositions of exponential operators (Phys. Lett. A, 1990).

Citizen Gardens © 2025 CC-NC-ND 4.0